



FOR JAVA DEVELOPERS MOVING TO NESTJS

Two pods. Every job twice.

Spring Boot to NestJS, episode 25: scheduled jobs and cron

A schedule says when a job may start. Not how many processes start it, whether runs overlap, or what happens when the one running it dies.

Two instances ran every tick twice, on Spring Boot and on NestJS: every process schedules every job.

ShedLock on Redis and Nest's `@OnOneInstance` with a Redis store stopped it. Nest's default in-memory lock store did not.

Killed mid-run, the job moved in about four seconds under a renewed 3 s lease and nearly ten under a fixed 10 s ShedLock maximum.

In scope

- `@Scheduled`, `@Cron`, `@Interval`
- ShedLock and `@nestjs/locks`
- Overlap, capacity, failures
- A real crash of the lock holder

Not in scope

- Queues and background workers (next episode)
- Kubernetes CronJobs

At the time of recording `@nestjs/locks` is version 0.0.1.

01 The measurements

A JOB EVERY SECOND, 3 s	6 runs, every tick twice, both stacks
@OnOneInstance, DEFAULT MEMORY STORE	still every tick twice
ShedLock / @OnOneInstance ON REDIS	3 runs, no tick twice
2.5 s JOB EVERY SECOND, ONE INSTANCE	Nest @Cron 3 at once; waitForCompletion 1; Spring 1
HEARTBEAT BESIDE AN 800 ms JOB	Spring pool 1: 920 ms gap; Node thread-holding: 1035 ms
KILLED 1 s INTO A 4 s JOB	lease 3 s: 4.0, 3.9 s; ShedLock 10 s: 9.7, 9.6 s

Two instances unless stated.

02 Lock the job in a shared store

Without a lock, every instance runs every tick. With one, at most one instance runs it while the lease holds. Not exactly once: a tick can still be missed.

Jobs.java

```
@Scheduled(cron = "* * * * *")
@SchedulerLock(name = "payments:settle", lockAtLeastFor = "500ms")
public void settleOnce() { ... }
```

jobs.ts

```
@Cron('* * * * *')
@OnOneInstance({ key: 'payments:settle' })
settleOnce() { ... }
```

two instances, Redis store: 3 runs in 3 s, no tick twice, on both

Run on a real machine, 2026-09-30, by `scripts/verify-schedule.sh` in the course repository.

03 Overlap

A plain Nest `@Cron` starts a new run every tick even while the last one runs. `waitForCompletion` stops that in one process; `@WithoutOverlapping` stops it through the lock.

jobs.ts

```
@Cron('* * * * *', { waitForCompletion: true })
async report() { ... }

@Cron('* * * * *')
@WithoutOverlapping({ key: 'payments:report' })
async reportAlone() { ... }
```

a 2.5 s job every second: plain `@Cron` 3 at once; either fix 1

Run on a real machine, 2026-09-30, by `scripts/verify-schedule.sh` in the course repository.

Before you trust a scheduled job

1. How many instances will run it?
2. Is the lock store shared by all of them?
3. Can a run overlap the last one?
4. What else shares its thread or event loop?
5. How long until another instance takes over after a crash?
6. Missed ticks are not caught up: process what is due

Everything else

 Site https://code-with-sam-dev.github.io	 Facebook https://www.facebook.com/codewithsamdev
 YouTube https://www.youtube.com/@CodewithSam-Dev	 GitHub https://github.com/code-with-sam-dev
 LinkedIn https://www.linkedin.com/company/code-with-sam-dev	 Buy me a coffee https://buymeacoffee.com/codewithsam
 x https://x.com/CodeWithSamDev	 Buy me sadza https://buymesadza.com/codewithsam
 TikTok https://www.tiktok.com/@codewithsamdev	Every measurement: scripts/verify-schedule.sh https://github.com/code-with-sam-dev/spring-to-node
 Instagram https://www.instagram.com/codewithsamdev	NestJS distributed locks https://docs.nestjs.com/reliability/locks

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. Redis is a trademark of Redis Ltd. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-09-30 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.